

MLE5004: Computer systems architecture

IV. Addressing Modes.

Contents

4.1	Address computation	2
4.2	FAR and NEAR addresses	4
4.3	Addressing modes	4
4.3.1	Computing the offset of an operand	4
4.3.2	Register addressing	6
4.3.3	Immediate addressing	6
4.3.4	Direct memory addressing	6
4.3.5	Base memory addressing (register indirect)	7
4.3.6	Base or Index plus Displacement addressing	7
4.3.7	Base and Index plus Displacement addressing	9
4.3.8	Scale factor	11
4.4	Segment Override Prefix	11

X86 instructions can have zero or more operands; for example, a return (RET) instruction from a procedure (subroutine) can have no operands or an immediate operand; a negate (NEG) instruction forms the 2s complement of a single operand, NEG AX; a move instruction has two operands MOV AX, BX, which moves the contents of register BX to register AX. In the x86 instruction set, the first operand listed is the destination operand; the second operand listed is the source operand.

The instruction set provides various methods to address operands. The main methods are: **register**, **immediate**, **direct**, **register indirect**, **base**, **index**, and **base combined with index**. A *displacement* may also be present. These and other addressing methods are presented in what follows with some examples.

The *source operand* can be obtained from the instruction as (i) immediate data, (ii) from a general-purpose register, (iii) from a memory location, or (iv) from an input/output (I/O) port. The source operand is normally unchanged by the instruction execution. The result of an instruction execution can be sent to a destination in (i) a register, (ii) a location in memory, or to (iii) an I/O port. **Memory-to-memory operations are not permitted with single instructions** — this type of operation is reserved for string instructions, which require that the source and destination locations be set up before the transfer takes place.

An instruction has the format shown below, where a label is an identifier for a line of code or a block of code, followed by a colon (:). The mnemonic is the name of the instruction, for example, *ADD*, *SUB*, *MOV*, etc. This is followed by zero to three operands — depending on the operation — separated by commas

1 LABEL: MNEMONIC OPERAND1, OPERAND2, OPERAND3

Some instructions require an operand to be a quadword (eight bytes), such as a dividend for a divide operation, or generate a quadword result, such as the product for a multiply operation. The dividend or product are contained in a concatenated register pair, such as *EDX:EAX*, where the colon specifies concatenation.

4.1 Address computation

Definition 4.1 (Address of a memory location). The number of consecutive bytes from the beginning of the RAM memory to the beginning of that memory location represent the address of the location.

An uninterrupted sequence of memory locations, used for similar purposes during a program execution, represents a segment. So, a segment represents a logical section of a program's memory, featured by its **basic address** (beginning), by its **limit** (size) and by its **type**. Both basic address and segment's size are represented on 32 bits.

For the family of 8086-based processors, the term segment has two meanings:

1. A block of memory of discrete size, called a *physical segment*. The number of bytes in a physical memory segment is
 - 64K for 16-bit processors
 - 4 gigabytes for 32-bit processors.
2. A variable-sized block of memory, called a *logical segment* occupied by a program's code or data.

We will call *offset* the address of a location relative to the beginning of a segment, i.e. the number of bytes between the beginning of that segment and that particular memory location. An offset is valid only if his numerical value (represented on 32 bits) does not exceed the segment's limit which it refers to.

We will call *address specification* a pair of a *segment selector* and an *offset*. A *segment selector* is a numeric value of 16 bits which uniquely selects the accessed segment and his features. A *segment selector is defined and provided by the operating system*.

In hexadecimal an address specification can be written as:

$$s_3s_2s_1s_0 : o_7o_6o_5o_4o_3o_2o_1o_0$$

In this case, the segment selector $s_3s_2s_1s_0$ shows a segment access which has the address $b_7b_6b_5b_4b_3b_2b_1b_0$ and a limit given by $l_7l_6l_5l_4l_3l_2l_1l_0$. The base and the limit are determined by the CPU after performing a segmentation process.

To have access to a specific location, the following condition must be met:

$$o_7o_6o_5o_4o_3o_2o_1o_0 \leq l_7l_6l_5l_4l_3l_2l_1l_0.$$

Based on such a specification the actual segmentation address computation will be performed as: $a_7a_6a_5a_4a_3a_2a_1a_0 := b_7b_6b_5b_4b_3b_2b_1b_0 + o_7o_6o_5o_4o_3o_2o_1o_0$ where $a_7a_6a_5a_4a_3a_2a_1a_0$ is the computed address (written in hexadecimal). The above address is called a *linear address*, (or segmentation address).

An address specification is also named a *FAR address*. When an address is specified only by the offset, we call it a *NEAR address*.

An example of address specification: 8 : 1000h.

To compute the linear address corresponding to the above specification, the CPU will do the following:

1. It will check if the segment with the value 8 was defined by the operating system and blocks the access if such a segment wasn't defined; (*memory violation error...*)
2. It extracts the Base address (B) and the segment's limit (L), for example, as a result we may have $B = 2000h$ and $L = 4000h$;
3. It verifies if the offset exceeds the segment's limit: $1000h > 4000h$? If so, then the access would be blocked.
4. It adds the offset to B and it obtains the linear address $3000h(1000h + 2000h)$. This computation is performed by the *ADR* component from *BIU*.

This kind of addressing is called segmentation and we are talking about the *segmented addressing model*.

When the segments start from address 0 and has the maximum possible size (4GiB), any offset is automatically valid and segmentation isn't practically involved in the address computation. So, having $b_7b_6b_5b_4b_3b_2b_1b_0 = 00000000$, the address computation for the logical address $s_3s_2s_1s_0 : o_7o_6o_5o_4o_3o_2o_1o_0$ will result in the following *linear address*:

$$a_7a_6a_5a_4a_3a_2a_1a_0 := 00000000 + o_7o_6o_5o_4o_3o_2o_1o_0$$

$$a_7a_6a_5a_4a_3a_2a_1a_0 := o_7o_6o_5o_4o_3o_2o_1o_0$$

This particular mode of using segmentation, used by most modern operating systems is called the **flat memory model**.

The x86 processors also have a memory access control mechanism called **paging**, which is independent of address segmentation. Paging implies dividing the *virtual memory* into pages, which are associated to the available physical memory. (1 page = 4 KB = 2^{12} bytes = 4096 bytes).

The configuration and control of segmentation and paging mechanisms is performed by the operating system. Of these two, only segmentation interferes with address specification, paging being completely transparent relative to the user programs.

Both addresses computation and the use of segmentation and paging mechanisms are influenced by the *execution mode* of the CPU, the x86 processors support the following (more important) execution modes:

- **real mode**, on 16 bits (using a memory word of size equal to 16 bits and having a limited amount of memory - 1 MB);
- **protected mode** on 16 or 32 bits, characterized by using paging and segmentation;
- **virtual mode**, allows running real mode programs together with protected ones;
- **long mode**, on 32 or 64 bits, where paging is mandatory while segmentation is deactivated.

In our course we will focus on the architecture and the behavior of x86 processors in **protected mode** on 32 bits.

The x86 architecture defines 4 **types of segments**:

- **code segment**, it contains instructions;
- **data segment**, it contains data with which instructions work;
- **stack segment**;
- **extra segment**, supplementary data segment.

Every program is composed by one or more segments of one or more of the types specified above. At any given moment, during run time, there is only at most one active segment of any type. Registers **CS** (Code Segment), **DS** (Data Segment), **SS** (Stack Segment) and **ES** (Extra Segment) contain the values of the selectors of the active segments, for each segment type. Thus, registers CS, DS, SS, and ES determine the starting addresses and the dimensions of the 4 active segments: code, data, stack and extra segments. Registers **above** and **GS** can store selectors pointing to other auxiliary segments without having predetermined meaning. Because of their use, CS, DS, SS, ES, FS, and GS are called **segment registers** (or selector registers). The EIP register contains

the offset of the current instruction inside the current code segment, this register being managed exclusively by BIU.

4.2 FAR and NEAR addresses

To address a location in the RAM memory two values are needed: one to indicate the segment and one to indicate the offset inside that segment. For simplifying the process of memory addressing, the CPU implicitly chooses, in the absence of other specification, the segment's address from one of the segment registers CS, DS, SS, or ES. The default choice of a segment register is made according to some rules specific to the instruction used.

An address for which only the offset is specified and the segment address being implicitly taken from a segment register is called a **NEAR address**. A NEAR address always points inside one of the four active segments.

An address for which the programmer explicitly specifies a segment selector is called a **FAR address**. Thus, a FAR address represents a complete address specification and can be specified in one of the following ways:

- $s_3s_2s_1s_0$: offset_specification where $s_3s_2s_1s_0$ is a constant;
- segment_register : offset_specification, where the segment registers is CS, DS, SS, ES, FS or GS;
- FAR [variable], where variable is of type *QWORD* and contains the 6 bytes representing the FAR address.

The internal format of an FAR address is: at the smallest address we have the offset, and at the higher address (the word following the current doubleword) we have the word that stores the segment selector.

The address representation follows the little-endian representation: the less significant part has the smallest address, and the most significant one has the higher address.

4.3 Addressing modes

Addressing modes provide different ways to access operands. A displacement is an 8-bit, 16-bit, or 32-bit immediate value in the instruction and is used to address memory. A base is the contents of a general-purpose register (GPR). The registers normally used for this purpose are (E)BX and (E)BP. An index is the contents of a GPR. The registers that are normally used for this purpose are (E)SI and (E)DI.

4.3.1 Computing the offset of an operand

For an instruction there are 3 ways to express a required operand:

- **register mode**, if the required operand is a register (`MOV EAX, 17`);
- **immediate mode**, the value of the operand is specified (`MOV EAX, 17`);
- **memory addressing mode**, if the operand is located somewhere in memory. In this case the operand offset is computed using the formula:

$$offset_address = [base] + [index \times scale] + [constant]$$

Thus the offset_address is obtained from the following (maximum) four elements, also shown in Figure 1:

- **as base**: the content of one of the registers EAX, EBX, ECX, EDX, EBP, ESI, EDI or ESP;
- **as index**: the content of one of the registers EAX, EBX, ECX, EDX, EBP, ESI or EDI;
- **scale**: to multiply the value of the index register with 1, 2, 4 or 8;
- **displacement**: the value of a numeric constant, on a byte, word or a doubleword.

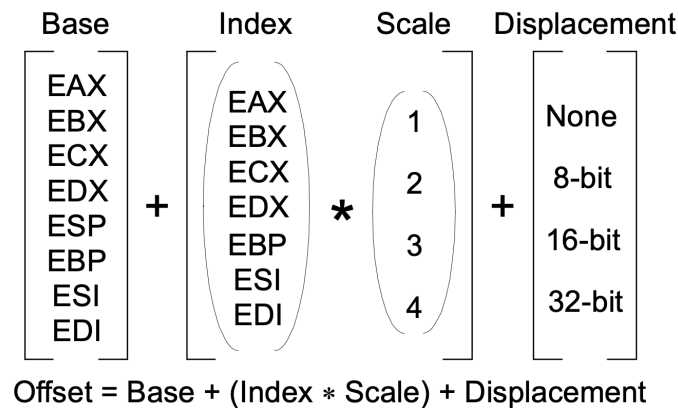


Figure 1: Offset (or Effective Address) Computation.

From here results the following modes to address the memory:

- *direct addressing*, when only the constant/displacement is present;
- *based addressing*, one of the base registers is present;
- *scale-indexed addressing*, one of the index registers is present.

These three mode of addressing can be combined (see the next subsections). A non direct addressing mode is called **indirect addressing** (based and/or indexed). So, an indirect addressing is one for which we have at least one register specified between squared brackets.

In case of the *jump* instructions another type of addressing is present called **relative addressing**. Relative addressing indicates the position of the next instruction to be run relative to the current position. This "distance" is expressed as the number of bytes to jump over. The x86 architecture allows *relative SHORT addresses*, described on a byte and having values between -128 and 127, but also relative NEAR addresses, represented on a doubleword with values between -2147483648 and 2147483647.

```

1  Jmp Below2 ; this instruction will be translated into (see OllyDbg) usually in
2  ;something as Jmp [0084]↓
3  % ...
4  ...
5  Below2:
6  Mov eax, ebx

```

4.3.2 Register addressing

Using register addressing, the instruction selects one or more registers which represent the operand or operands. This is the most compact addressing method, because the register addresses are encoded in the instruction. It achieves fast execution since the operation is performed entirely within the central processing unit (CPU); that is, there are no bus transfers to or from memory. Examples of register addressing are shown below, where the semicolon indicates a comment.

```

1 INC BH ;increment register BH
2 MOV AX, BX ;move contents of register BX to register AX
3 SUB EBX, EBX ;subtract EBX from EBX (EBX = 0)
4 XOR EAX, EAX ;exclusive-OR EAX with EAX (EAX = 0)

```

The source and destination registers can be any of the general-purpose registers, either 8-bit, 16-bit, or 32-bit GPRs. The segment registers CS, DS, SS, ES, FS, or GS can also be used in register addressing as well as the flag register and the floating-point registers, among others. The length of the operand is determined by the name of the GPR.

4.3.3 Immediate addressing

Some instructions have a data value encoded in the instruction so that it is available immediately as the source operand. The immediate data can be 8 bits, 16 bits, or 32 bits, which are sign-extended to fit the size of the destination operand. Examples of immediate addressing are shown below for both positive and negative values.

```

1 ADD EAX, 3 ;add 3 to register EAX
2 MOV AH, 00 ;reset AH; AH = 0000 0000
3 MOV EAX, 0ABCDEFH ;32-bit operand ABCD FFFF(Hex) = EAX
4 MOV AX, 302 ;AX = 0000 0001 0010 1110
5 MOV AX, -40 ;AX = 1111 1111 1101 1000

```

4.3.4 Direct memory addressing

The memory operand can be either the source or the destination operand and is addressed by a segment selector and a 32-bit offset that is part of the instruction. The combination of the selector and the offset generates an effective address that points directly to the memory operand. The segment selector can be any of the following segment registers: code segment (CS), data segment (DS), stack segment (SS), or the extra segment (ES).

A segment selector is loaded with the appropriate value, then the CPU addresses memory using the current (implicit) selector and the offset. Examples of direct memory addressing using an implicit segment selector are shown below.

```

1 MOV AL, [1A33D4H] ;AL = contents of memory at 1A33D4H
2 INC DWORD PTR [17H] ;incr 32-bit operand at offset 17H

```

4.3.5 Base memory addressing (register indirect)

Base, or register indirect, addressing is used when a register contains the address of the data, rather than the actual data to be accessed. This method of addressing uses the general-purpose registers within brackets; for example, [BX], [BP], [EAX], [EBX], etc., where the brackets specify that the indicated register contains an offset that points to the data within a specific segment. Registers (E)BX, (E)SI, and (E)DI contain offsets for processing operands in the data segment; for example, DS:BX, DS:ESI, and DS:DI. Register (E)BP is used to reference data in the stack segment, such as SS:EBP.

Examples of indirect memory addressing are shown below. In the first example, the notation [[EBX]] in the comments reads “the contents of the contents of EBX”; that is, the contents of memory specified by the contents of EBX are loaded into register ECX. In the second example, assume that BX = 0002H and that memory locations 0002H and 0003H contain 00H and FFH, respectively. Then register AX = FF 00H after completion of the move instruction, where 00H is the low-order byte in AL. This is illustrated in Figure 2.

```

1 MOV ECX, [EBX] ;ECX = [[EBX]];
2 % ;If [EBX] = 1000H, then ;ECX = bytes [1000H through 1003H]
3 MOV AX, [BX] ;AX = [0002H] , [0003H]
```

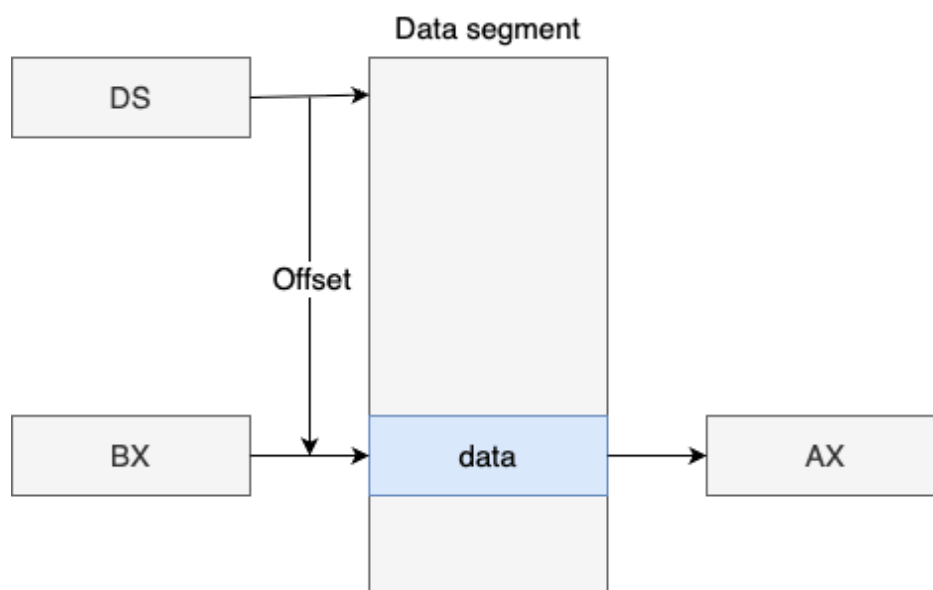


Figure 2: Indirect memory addressing.

4.3.6 Base or Index plus Displacement addressing

This is a variation of base addressing and uses the general-purpose registers (E)AX, (E)BX, (E)CX, (E)DX, (E)SP, (E)BP, (E)SI, or (E)DI as the base address or (E)AX, (E)BX, (E)CX, (E)DX, (E)BP, (E)SI, or (E)DI as an index address, plus a displacement to access memory. A scaling factor may also be used in conjunction with the index register. The displacement consists of a signed number in 2s complement representation that is added to the value in the base register or index register to produce an effective address. When (E)BP is utilized as the base register, the stack segment is the default segment.

An expression using a displacement is usually written as $[BX + 4]$ using BX as the base register with a displacement of 4, or as $[ESI + 8]$ using ESI as the index register with a displacement of 8. Both methods indicate a displacement of four or eight bytes from the base or index address stored in register BX or register ESI. This addressing method can be used as an index into an array, as shown in the code segment shown below, where the DB directive defines the five values as bytes. The compare instruction compares the contents of AL with the byte at location $TABLE + 4$, which is the fifth byte (4) of the list. The first element (15) of the list is at location $TABLE$, which has a displacement of zero. Changing the value of the displacement accesses different fields in the same record. Changing the base register accesses the same field in different records.

```

1  TABLE: DB 15, 7, 6, 10, 4
2  ...
3  LEA BX, TABLE ;loads the effective
4  ;addr of TABLE into BX
5  CMP AL, [BX + 4]

```

Base or index addressing provides a way to address tables, arrays, or lists which may be located in different areas of memory. A base register can be set to point to the base of a table. The elements of the table can then be addressed by their displacement from the base. Different tables can be addressed by simply changing the value in the base register. A graphical representation of this concept is illustrated in Figure 3, which shows relevant information regarding an employee. The contents of register BX can be changed to access a different employee, where the same displacement addresses the rate.

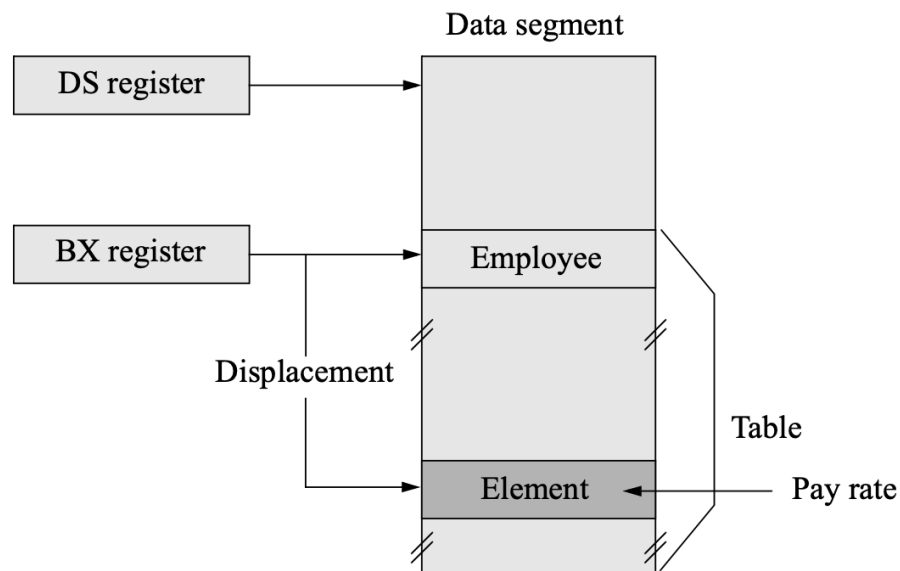


Figure 3: Base addressing with displacement.

Alternatively, this addressing method can be used to access elements in an array. The displacement value locates the beginning of the array and the value stored in the index register selects an element in the array, as shown in Figure 4.

The effective address in Figure 4 is obtained from the sum of the displacement and the contents of index register ESI. Indexed addressing is used to access elements in an array at execution time.

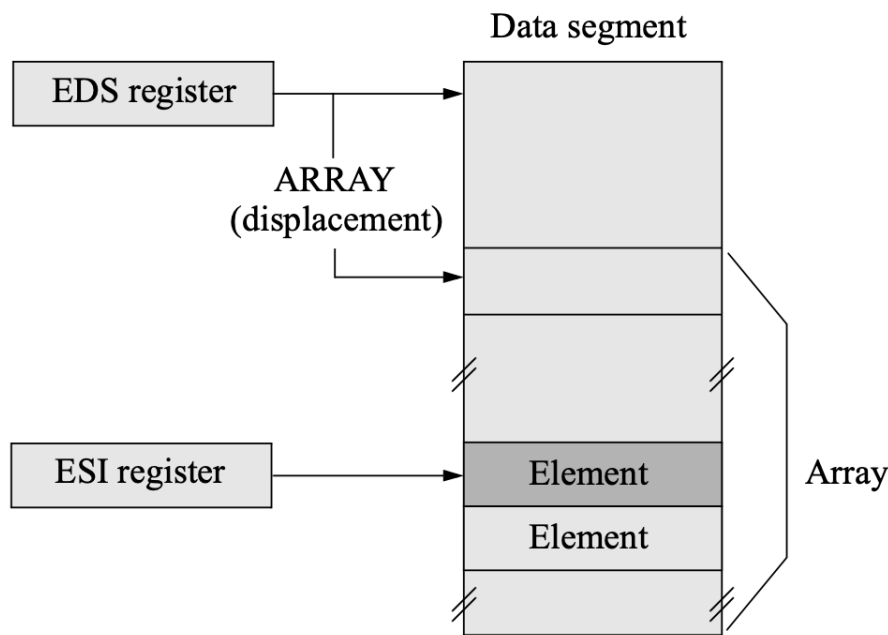


Figure 4: Index addressing with displacement.

The value of the index register selects an element in the array; the first element is selected if the index register contains a value of zero. The elements are the same length; therefore, simple arithmetic on the index register will select any element. An instruction to move the contents of an array element to register EAX is shown below, where ARRAY is the address (displacement) of the array (or table) and the element is accessed whose index is in ESI.

```
1  MOV EAX, ARRAY[ESI]
```

ARRAY[ESI] references the contents of the element in the location addressed by the displacement ARRAY plus the contents of ESI — $\text{ARRAY} + 0$ references the first element in the array. To access the next element, simply increment register ESI.

4.3.7 Base and Index plus Displacement addressing

The effective address of the operand is obtained by adding the contents of the base register, the contents of the index register, and the displacement (which may be zero). Base indexed addressing without displacement is frequently used to access the elements of a dynamic array. A dynamic array is an array whose base address can change during program execution. This addressing method can also be used to access a 2-dimensional array in which the beginning of the array is determined by the displacement and the base register; the index register addresses the array elements.

Another application is to access elements in a 1-dimensional array on a stack, as shown in Figure 6. The base register can point to the stack top; the contents of the base register plus the displacement points to the beginning of the array. The index register then selects elements in the array. The effective address (EA) is calculated as shown below.

Example 1.

1 $EA = EBP + \text{displacement} + ESI$

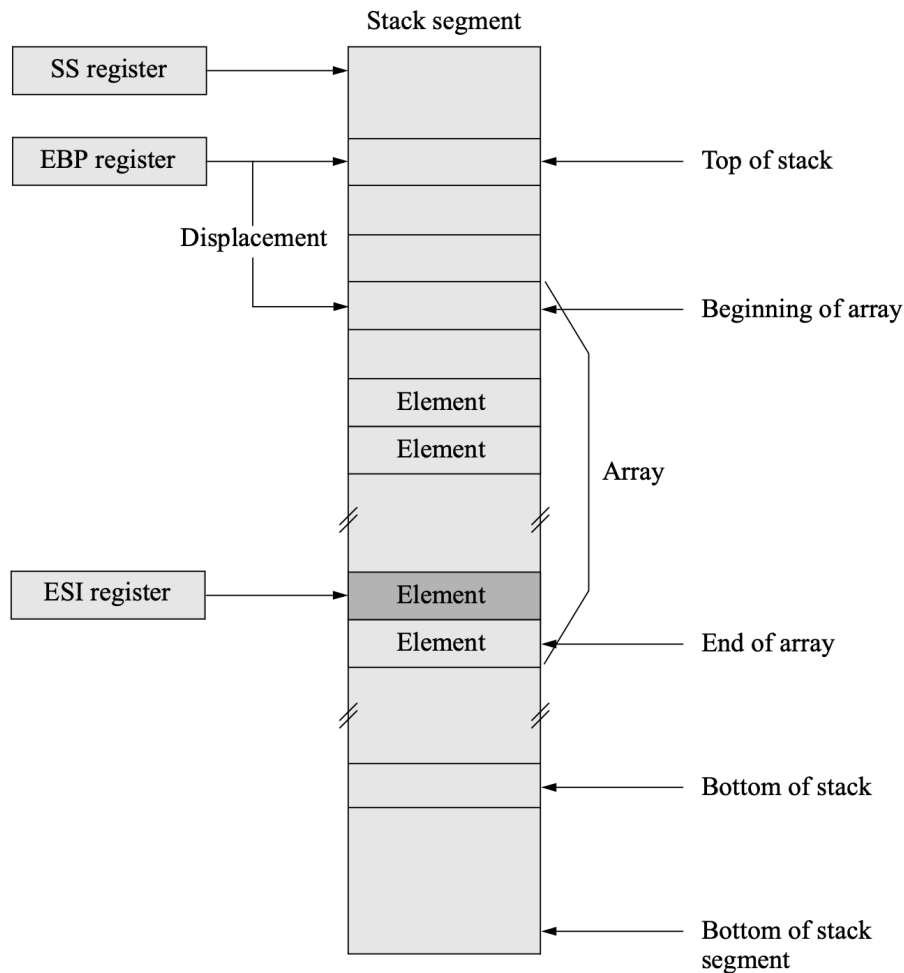


Figure 5: Base and index addressing with displacement.

A 2-dimensional array is shown in Table 1 in which the elements are stored row by row. The elements can be accessed by base-index with displacement addressing. The second element in the first row (56) can be accessed by by storing the base address of the first element in BX/EBX (or

1 $[EBX/EBP] [ESI/EDI]$

BP/EBP) and storing a 1 in SI/ESI (or DI/EDI) with no displacement — the index register selects the i th element. A 1 rather than a 2 is stored in the index register, because the element whose address is in the base register is the first element (12). To access the second element in the second row (89), a value of 5 is added to the base register — the number of elements in a row — as shown below assuming a value of 1 in the index register. or

1 $[EBX/EBP] [ESI/EDI]$

Table 1: An illustration of a 2-dimensional array.

12	56	78	5	44
54	89	65	92	6
37	66	15	77	19

1 `[EBX/EBP + ESI/EDI + 5]`

4.3.8 Scale factor

A scale factor can be used to access elements in an array in which the size of the elements is two, four, or eight bytes. The address calculation can be considered as

$$(\text{index register} * 2^{\text{scale}}) + \text{displacement}$$

or

$$\text{base register} + (\text{index register} * 2^{\text{scale}}) + \text{displacement}$$

where the scale is 0, 1, 2, or 3. This addressing mode is useful for processing arrays and provides an index into the array when the array elements are two, four, or eight bytes. If the array elements are 8-byte operands, then the index register is multiplied by eight prior to calculating the effective address. Since the scale factor is a power of two, multiplication can be accomplished by shifting left rather than by a multiply operation.

(Index times scale) plus displacement The displacement points to the beginning of the array, while the index contains the address of the appropriate array element; that is, the index is the subscript of the array element. The effective address of the operand is obtained by multiplying the index by a scale factor of one, two, four, or eight and then adding the displacement.

Base plus (index times scale) plus displacement The effective address of the operand is obtained by multiplying the index register by a scale factor of one, two, four, or eight, then adding the base register, and then adding the displacement. This addressing mode is useful for accessing array elements in a 2-dimensional array when the operands are one, two, four, or eight bytes. An example is shown in Figure ?? using EBX as the base register, ESI as the index register with a scale factor of 4, and a displacement for the following move instruction that moves the addressed operand into register EAX:

1 `MOV EAX, [EBX + ESI * 4 + DISPLACEMENT]`

4.4 Segment Override Prefix

The processor selects the applicable default segment as a function of the instruction: instruction fetching assumes the code segment; accessing data in main memory references the data segment; and instructions that pertain to the stack reference the stack segment. When transferring data to

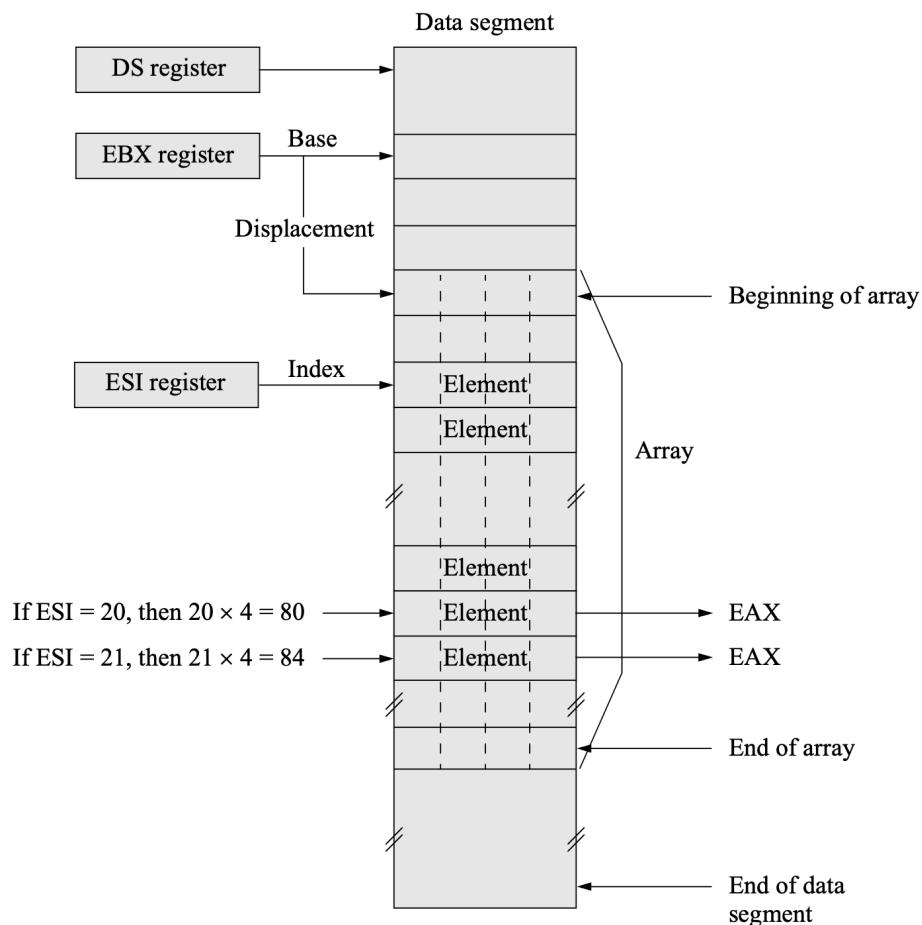


Figure 6: Base plus (index times scale) plus displacement addressing example.

or from memory, the processor usually references the current data segment as the default segment. However, a segment override prefix can be used to change the default data segment to another segment; that is, to explicitly specify any segment register to be used as the current segment.

This is accomplished by listing the new segment to be used for source or destination operands followed by a colon and is placed in front of the offset variable. For example, to override the data segment with the extra segment, the coding shown below applies to a move instruction, which moves the 32-bit operand addressed by register EBX in the extra segment to register EAX. When the move instruction is executed, the offset in register EBX pertains to the extra segment rather than the data segment. The segment override prefix is a byte that is inserted in the instruction and

```
1 MOV EAX, ES:[EBX]
```

causes the processor to fetch the memory operand from the designated segment rather than from the default segment. Some segments, however, cannot be overridden. Instruction fetches must be made from the code segment; push and pop operations must be made with reference to the stack segment; and string operations must provide a destination address in the extra segment.

The segment override prefix applies only to the instruction in which it is inserted; that is, it is restricted to a single instruction - all subsequent instructions use the default segment. Thus, the segment override prefix must be specified for each instruction that is to be overridden. Utilizing a

segment override prefix increases the execution time of the corresponding instruction.

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press.
<https://doi.org/10.1201/b14582>
3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.